東北大學 NORTHEASTERN UNIVERSITY

搜 索

罗 子 涵

东 北 大 学 ACM 校 队

目录

1 搜索的介绍

2 广度优先搜索 BFS

3 深度优先搜索 DFS

4 搜索的优化

什么是搜索？

搜索在日常及编程领域应用广泛，如搜索引擎和文件搜索。尤其在算法竞赛中，当遇到无法通过数学模型或现有算法直接解决、必须遍历所有可能才能得出结果的问题时，搜索便成为不可或缺的解题手段。鉴于其在各类竞赛中的重要性，深入理解搜索的分类及其应用技巧，对于高效解决问题和提升编程能力至关重要。

> 搜索是一种通过穷举所有可能的解的状态，来求得题目所要求的解或者最优解的方法，即通过枚举所有的可能状态，来寻求一个解或者最优解。
>
> —《算法基础训练教程》

什么是广度优先搜索 (BFS)？

算法核心：从起始节点出发，首先访问所有与其距离为 1 的节点，然后是距离为 2 的节点，依此类推，按照“从近到远”的顺序逐层遍历所有可达节点。BFS 通常借助队列实现，能够在无权图中找到从起点到终点的最短路径。属于**盲目搜索**。

广度优先搜索的基本原则：先将起点加入队列，然后不断取出队首元素，访问其所有未被访问过的相邻节点，并将这些节点加入队列。直到队列为空，表示所有可达节点均已被访问。

BFS 算法的抽象

问题描述：

给定一棵树，从根节点开始，按照广度优先的顺序访问所有可达节点，并输出访问的顺序。

伪代码：

```
创建空队列 q
将 root 加入队列 q
当 q 非空时:
  取出队首元素 u
  输出 u
  对于 u 的每个子节点 v:
    将 v 加入队列 q
```

BFS 简单举例

题目描述：
数字华容道是一个经典的益智游戏，它的目标是在一个 3×3 的棋盘上，通过不断地将数字移动到空位（用 0 表示）上，最终将棋盘排列成如下形式：

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{bmatrix}$$

每次只能将数字移动到相邻的空格中（上下左右方向）。

问题：
给定一个初始局面，求将其变为目标局面所需的最小移动步数；如果无法达成目标，则输出 -1。

什么是深度优先搜索 (DFS)?

算法核心：沿着树的深度遍历树的节点，尽可能深的搜索树的分支。当节点 v 的所有边都被探寻过，搜索将回溯到发现节点 v 的那条边的起始节点。这一过程一直进行到已发现从源节点可达的所有节点为止。如果还存在未被发现的节点，则选择其中一个作为源节点并重复以上过程。整个进程反复进行直到所有节点都被访问为止。属于**盲目搜索**。

深度优先搜索的基本原则：按照某种条件往前试探搜索，如果前进中遭到失败 (正如走迷宫遇到死路)，则退回头另选通路继续搜索，直到找到满足条件的目标为止。

DFS 算法的抽象

问题描述：

给定一棵树，从根节点开始，按照深度优先的顺序访问所有可达节点，并输出访问的顺序。

伪代码：

```
函数 dfs(节点 u):
  输出 u
  对于 u 的每个子节点 v:
    调用 dfs(v)
```

DFS 简单举例

题目描述：

你的任务是在一个国际象棋棋盘上放置八个皇后，使得任意两个皇后之间都不存在攻击关系。

棋盘上的每个格子要么是空闲的，要么是预留的；你只能将皇后放置在空闲格子上。

需要注意的是，预留格子不会阻挡皇后的攻击路径。

问题：

给定一个包含预留格子的 8×8 棋盘，有多少种合法方案可以放置八个皇后？

0-1 BFS

算法核心：
0-1 BFS 是一种针对边权只有 0 或 1 的图的最短路算法，是堆优化的 Dijkstra 算法的优化特例。它使用双端队列（deque）代替普通队列或优先队列，确保在每一步总是优先处理“更近的点”，从而达到高效的最短路径更新。

优化思想：
与传统 BFS 不同，0-1 BFS 会将权重为 0 的边对应的新节点插入队首，将权重为 1 的边对应的新节点插入队尾。这样保证了队列中的元素始终按照“代价从小到大”的顺序被访问，从而实现类似于 Dijkstra 的最短路径计算，但效率更高。

适用范围：边权只包含 0 和 1 的稀疏图。
实现复杂度：时间复杂度 $O(n+m)$

DFS 回溯

基本思想：回溯法是一种在搜索树中系统地搜索解的策略，适合解决排列、组合、子集、划分等问题。其本质是 DFS + 状态恢复，即“做选择 → 递归探索 → 撤销选择”。

伪代码：

```
函数 backtrack(路径, 选择列表):
  如果满足结束条件：
    将路径加入 result
    返回
  遍历每个选择 in 选择列表:
    做选择（将其加入路径，并从选择列表中移除）
    递归调用 backtrack(路径, 新的选择列表)
    撤销选择（将选择移出路径，恢复选择列表）
```

自强不息·知行合一 NEU

题目描述：
要求给定一个包含数字的集合，返回所有可能的排列。每个排列是这个集合中所有数字的一个独特排列，并且排列顺序可以不同。

例如：
对于 $nums = [1, 2, 3]$ 它的所有可能的排列为 $[1, 2, 3]$, $[1, 3, 2]$, $[2, 1, 3]$, $[2, 3, 1]$, $[3, 1, 2]$, $[3, 2, 1]$

DFS 回溯

```
void dfs(int step) {
    if (step == n) {
        for (int i = 0; i < n; i++) cout << path[i] << " \n"[i == n - 1];
        return;
    }
    for (int i = 0; i < n; i++) {
        if (vis[i]) continue;
        vis[i] = true;
        path.push_back(nums[i]);
        dfs(step + 1);
        visited[i] = false;
        path.pop_back();
    }
}
```

迭代加深 DFS

算法原理：
通过将深度限制从 1 开始逐步增加，模拟广度优先搜索的行为，同时避免了 BFS 的空间开销。

核心思想：
从深度为 1 的 DFS 搜索开始，逐步增加搜索深度，进行深度为 2、3、4 等的 DFS 搜索，直到找到目标或遍历完所有可能的节点。

优缺点：

- 优点：
  - 节省空间，适合在空间有限的环境下使用。
  - 减少空间开辟所需的时间常数，提高效率。
  - 保证找到最短路径。
- 缺点：每次增加深度时会重新访问同一节点，效率较低，尤其是在状态存储较为简单的情况下。

题目描述： 数字华容道的升级版，棋盘大小提升为 4×4。

核心代码：

```
bool dfs(int maxn, int step, int w, int x, int y) {
    if (check()) return true;
    if (step > maxn) return false;
    for (int i = 0; i < 4; i++) {
        if (i == (w ^ 1)) continue;
        int nx = x + dx[i], ny = y + dy[i];
        if (nx < 1 || nx > 4 || ny < 1 || ny > 4) continue;
        swap(map[x][y], map[nx][ny]);
        if (dfs(maxn, step + 1, i, nx, ny)) return true;
        swap(map[x][y], map[nx][ny]);
    }
    return false;
}

void solve() {
    int step = 0;
    while (!dfs(step, 0, -1, sx, sy)) step++;
    cout << step << endl;
}
```

迭代加深 DFS 真的更有优势吗？

假设搜索树是一棵深度为 n 的满 k 叉树，存储一个状态需要常数空间 m。

为了便于推导等比数列公式，这里取 $k = 2$。

时间复杂度：

- BFS：$m \times (1 + 2^1 + 2^2 + \cdots + 2^{n-1}) \approx m \times 2^n$
- DFS（迭代加深）：$2^1 - 1 + 2^2 - 1 + \cdots + 2^n - 1 \approx 2^{n+1}$

空间复杂度：

- BFS：$O(m \times 2^n)$
- DFS（迭代加深）：$O(m + n)$

搜索的剪枝

算法原理：
通过提前排除不必要的搜索路径，减少遍历无效或不符合条件的节点，从而提高算法的效率。剪枝既可以用于 DFS，也可以用于 BFS。

关键点：
要对后续节点进行合理的预判，过度剪枝可能会导致遗漏正确的解或答案。

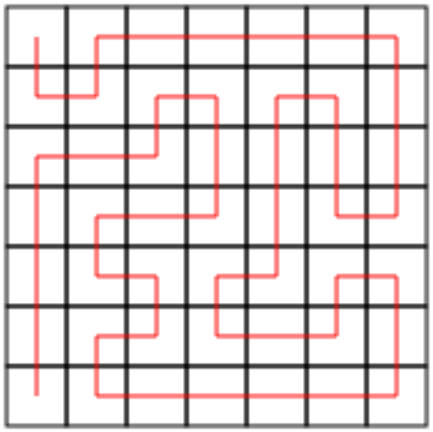
常见方法：

1. 提前预判状态的可行性
2. 记忆化搜索

题目描述：

在一个 7×7 的网格中，从左上角出发到左下角，共有 88418 条不同路径。D（向下）、U（向上）、L（向左）、R（向右）。

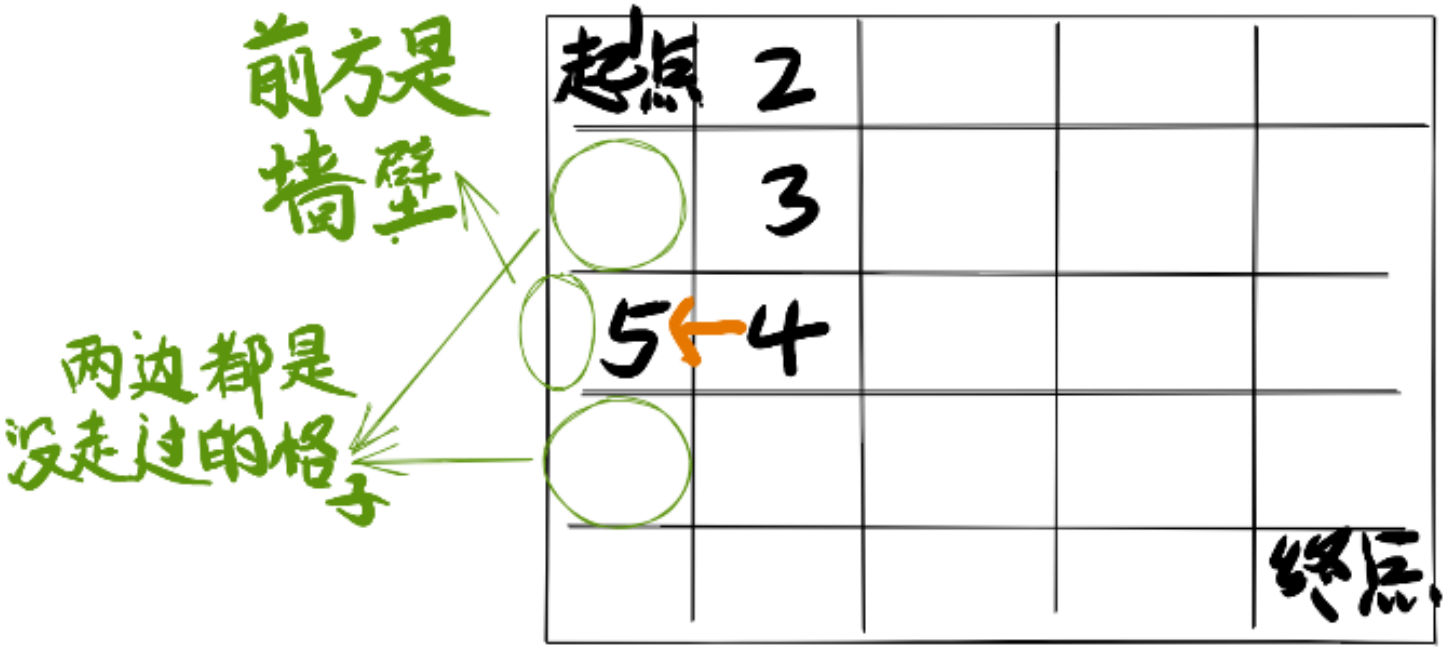
例如，图中路径对应的描述为：



DRURRRRRDDDLUULDDDLDRRURDDLLLLLURULURRUULDLLDDDD

现在，给定一个可能包含通配符？的路径描述，表示某些方向不确定，请你计算与该描述匹配的、有效的不重复路径数量。

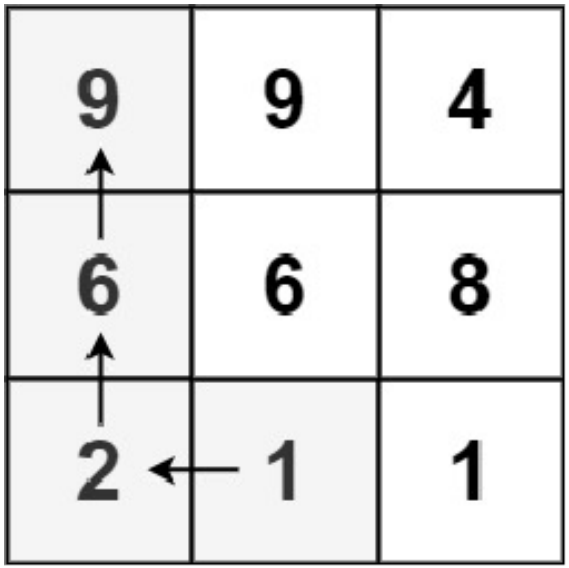
减枝方法：



题目描述：

给定一个 $m \times n$ 的整数矩阵 `matrix`，请你找出其中 **最长递增路径**的长度。

从任意单元格出发，每次可以向 **上**、**下**、**左**、**右**四个方向移动，不能沿对角线移动，也不能移出矩阵边界（不允许环绕）。



输入：matrix = [[9,9,4],[6,6,8],[2,1,1]]

输出：4

解释：最长递增路径为 [1, 2, 6, 9]。

双向搜索

基本思想：
从起点和终点分别同时进行搜索，在中间某一步相遇，从而完成整体路径的构造或判定。

适用场景：

- 起点和终点已知，目标是寻找可行路径或最短路径；
- 状态具有对称性，正反过程逻辑相近。

优势：

- 指数级问题中显著减少搜索层数；
- 减少重复计算，降低时间与空间复杂度；

问题描述：

给定一个 $N \times N$ 的网格，每个格子 (i, j) 上写有一个非负整数 $a_{i,j}$。你从左上角 $(1, 1)$ 出发，每次只能向右或向下走一步，目标是到达右下角 (N, N)。

要求统计有多少条路径，使得路径上所有经过的格子中的数的按位异或和为 0。路径包括起点和终点。

输入约束：

- $2 \leq N \leq 20$
- $0 \leq a_{i,j} < 2^{30} \quad (1 \leq i, j \leq N)$
- 所有值均为整数

请提问

自强不息·知行合一 NEU

;